System overview

TBK SPEC 01:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 01:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

What tebako is 4

Core capabilities 4

Repositories and languages 5

tebako-rs crate roles (MECE, open/closed) 6

Per-language tpkg bindings 6

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

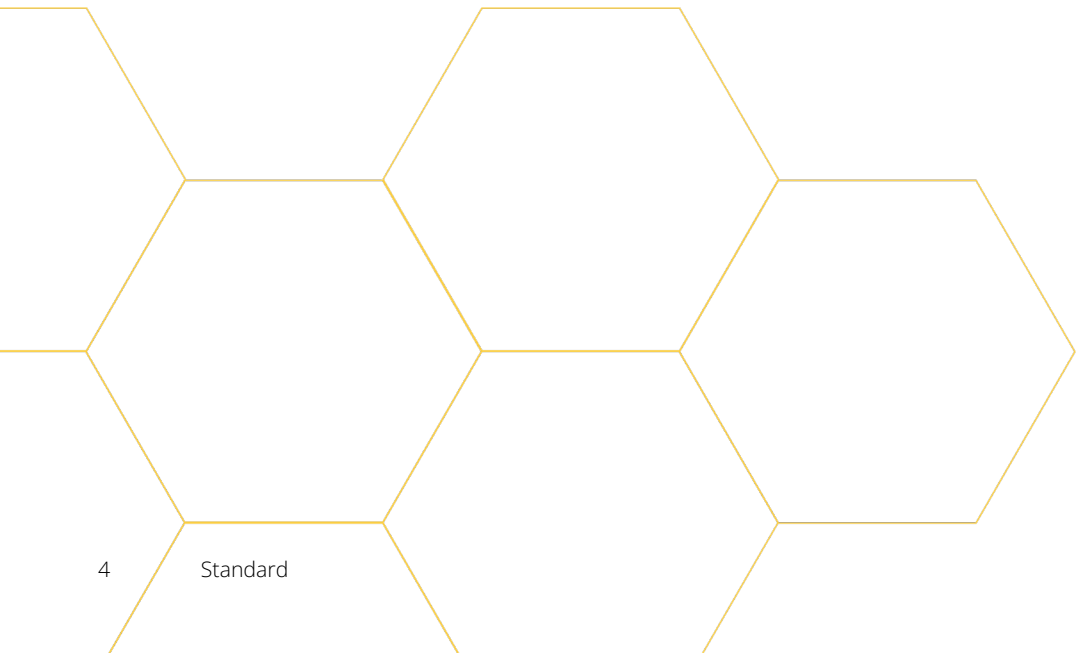
What tebako is

Tebako is a **packaging and loading ecosystem** — not a Ruby tool. Ruby is the first runtime instance. The system is generic across four axes:

- **Any runtime.** A runtime is a payload that *provides* an interpreter (ruby today; python, julia, qtk, anything later). The loader neither knows nor cares what language it starts.
- **Any platform.** Artifacts are per platform triplet (spec 03 §platform axis); payloads may also be `universal`.
- **Any payload.** Application code, data, fonts, native tools, runtimes themselves — all images of the same kind, differing only in what their manifests declare.
- **Any composition.** Payloads stitch onto a single binary, and payloads recursively mount other payloads at paths (spec 11 §encapsulation).

Core capabilities

1. **Stitch** — N payload images + bootstrap into one executable (fat), or a small executable resolving parts on demand (lean).
2. **Load** — mount payloads (recursively) into one VFS; transfer control to the declared entrypoint.
3. **Share** — runtimes and payloads download once into a machine-wide content cache; every package on the machine reuses them.
4. **Shim** — tebako manages shims for EVERY executable every installed payload provides; one payload may provide MULTIPLE executables, each becoming a registered command (spec 07).
5. **Jail** — declarative host-filesystem policy per run, enforced by the VFS layer (spec 08).



© 2026 The Tebako Project

6. **Trust** — opt-in per package: authentication, signatures, encryption (specs 09, 10). Unsigned stays first-class.

Repositories and languages

Tebako-owned C/C++ exists in exactly **one** repository: `dwarfs-t`. (The `tamatebako/ruby` fork carries upstream ruby's own C source plus patches — vendored upstream, not ours; `tebako-runtime-ruby` is build orchestration that links the Rust staticlibs `tfs` + `tebako-driver`.) Everywhere else: Rust, pure Ruby, or Docker. Owner-locked.

TABLE 1

Repo	Language	Role
`tamatebako/ruby`	Ruby tooling + C (ruby)	Patch sets (canonical `.patch` files + `versions.yml`); release = patched source tarballs per version × scenario. The *source factory*.
`tebako-runtime-ruby`	Ruby builder + C/CMake	Builds runtime payloads per version × triplet; release = interpreters + `.tfs` images + index. The *runtime factory* (first instance; python/julia factories follow the same shape).
`tamatebako/dwarfs-t`	C++	The DwarFS-T image format: `libdwarfs_c` reader + creation-time Writer. The only surviving C++ library; read-only at runtime forever.
`tamatebako/dwarfs-rs`	Rust (FFI)	`dwarfs-t` / `dwarfs-t-sys` safe bindings to `libdwarfs_c`. Standalone by design — not tebako-specific.
`tamatebako/tebako`	Rust only	**The product**: packager, loader, VFS, formats, trust. (The repo-swap 2026-07-27: formerly `tebako-rs`; the C++/gem era is archived at `tamatebako/tebako-v1`.)
`tebako-ci-containers`	Dockerfile	Toolchain images for the three factories.
`tebako.org`	AsciiDoc/Astro	Docs, blog, trust-anchor publication.
`tamatebako/tebako` (gem)	Ruby	RETIRED as orchestrator (2026-07-26); maintenance mode. Endgame: a ~100-line pure-Ruby installer shim that fetches the platform binary (no FFI), or full archival.
`tamatebako/libtfs` (C++)	C++	LEGACY; superseded by `crates/tfs` (164-test contract parity).

© 2026 The Tebako Project

tebako-rs crate roles (MECE, open/closed)

TABLE 2

Crate	Kind	Role
`tpkg`	lib	L0 wire format, single source of truth (spec 02). `#![forbid(unsafe_code)]`.
`tfs`	lib (cdylib+staticlib +rlib)	The Rust TFS (spec 11): `tebako_fs_*` C ABI, mount table, dispatch, backends ZIP/DwarFS/SquashFS. Sole home of future COW/ENC transforms.
`sqfs-sys`	lib (FFI)	libsquashfs-tools-ng FFI, ABI-pinned.
`tebako-pkg`	bin	Container surgery: `info/bundle/unbundle/reassemble/insert-image/remove-image/set-runtime/sign/verify`.
`tfs-cli`	bin	The `tfs` image tool: `info/ls/tree/cat/stat/extract/find/mkimage` (in-process Writer).
`tebako-cli`	bin	The packager: `press` (lean/fat), `cache list/prune`.
`tebako-bootstrap`	bin	The loader (spec 06): trailer parse, resolution, verification, ABI-v1 handoff. < 3 MB static, size-gated (unverified-first: rnp/botan feature-gated out; OpenPGP moves to the roadmap-72 crypto toolkit).
`tebako-driver`	lib (staticlib+rlib)	The spec-17 runtime driver: mounts the env image + payload images into the process-global TFS context, applies `TEBAKO_JAIL`, rewrites argv to the entrypoint. Linked into every runtime executable. The Rust successor of the v1 C++ `tebako-main`.
`tebako-http`	lib	In-process HTTPS (ureq+rustls, webpki-roots bundled; OS roots opt-in).
`tebako-signer`	lib	OpenPGP via `rnp-rs` (crates.io, `vendored`); successor-key rotation.
`tests/contract`	tests	The parity oracle: the C++ c_api suite through the Rust ABI (164 tests).

Per-language tpkg bindings

The tpkg contract is manipulated per language — never by shelling a foreign binary: `crates/tpkg` (Rust), `tpkg.h` (C, cbindgen — lands with its first real C consumer), `tpkg.rb` (Ruby, formalized from the gem's trailer reading).

© 2026 The Tebako Project